Project Design Phase

Solution Architecture

Date	29 October 2026
Team ID	
Project Name	FreelanceHub — MERN Stack Freelancing Platform
Maximum Marks	4 Marks

Solution Architecture

Solution architecture is the process that bridges the gap between a business problem and its technical implementation. For FreelanceHub, that means translating the problems identified in the Ideation Phase — freelancers needing a clear way to find work, apply and track their applications, and clients needing a simple way to post projects, compare proposals and collaborate with the freelancer they hire — into a concrete, working technical structure.

Its goals for this project are to:

- Define the technology stack best suited to solving the identified problems (React + Vite, Express, MongoDB, Socket.io).
- Describe the structure, layers, and data flow of the application to any stakeholder reviewing the project.
- Define how each architectural layer maps to a specific feature and business requirement.
- Provide a reference that guided implementation from setup through to the completed application.

FreelanceHub Solution Architecture Diagram

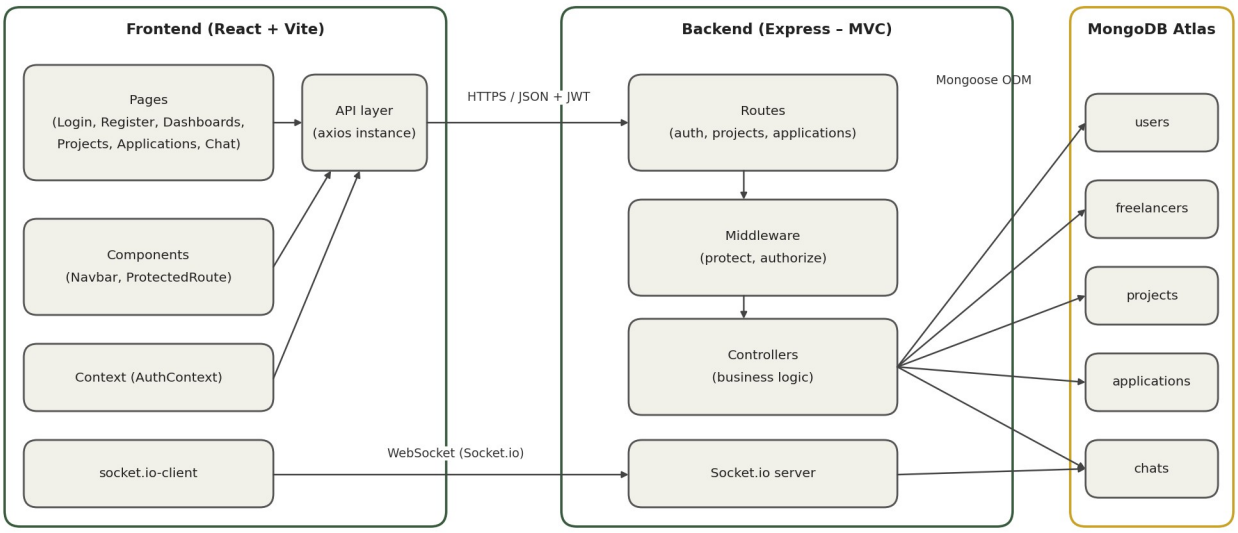


Figure 1: FreelanceHub solution architecture — presentation, application, and data layers

How Each Layer Solves the Business Problem

- **Presentation Layer (React + Vite):** Delivers the registration and login screens, the client dashboard for posting projects, the freelancer pages for browsing, filtering and applying, the application status views and the chat window — directly addressing the need for clarity and speed. Auth Context and ProtectedRoute make sure each user only sees the pages their role allows.
- **Application Layer (Express — Routes, Middleware, Controllers):** Enforces business rules that a purely client-side app could not guarantee — hashing passwords with bcrypt, verifying the JWT on every protected request, checking the user's role (for example, only clients can post projects and only freelancers can apply), and updating application status when a client approves or rejects.
- **Real-time Layer (Socket.io):** Keeps an open connection between client and server so that, once an application is approved, the client and freelancer can exchange messages instantly in a shared room, with every message saved to the database.
- **Data Layer (MongoDB Atlas):** Stores users, freelancer profiles, projects, applications and chats as separate Mongoose-modelled collections linked by references, so a project, the applications it receives and the chat that follows an approval all stay connected in one data model.